Curso
Tendencias Aplicadas de Ingenería de Software

Redis

Redis: Introducción

- Redis (Remote Dictionary Server) es una base de datos NoSQL y utilizada en general para operar en memoria principal (RAM).

- A diferencia de los sistemas de gestión de bases de datos relacionales tradicionales, Redis adopta un modelo clave-valor.

- Cada clave puede almacenar distintos tipos de estructuras de datos como cadenas de caracteres, listas, conjuntos, tablas hash, conjuntos ordenados y flujos de eventos.

Motivación para Utilizar Redis

- Los sistemas modernos deben procesar cantidades crecientes de usuarios, transacciones y solicitudes concurrentes. En muchos casos, las bases de datos relacionales se convierten en un cuello de botella debido al costo de acceder al almacenamiento persistente.
- Redis surge como una solución para reducir este problema mediante el almacenamiento temporal de información frecuentemente utilizada.
- Al mantener los datos en memoria RAM, las operaciones de lectura y escritura pueden ejecutarse varios órdenes de magnitud más rápido que en sistemas basados en disco.

Motivación para Utilizar Redis

- Los principales problemas que Redis ayuda a resolver son:
  - Latencias elevadas en consultas frecuentes.
  - Sobrecarga de bases de datos relacionales.
  - Gestión eficiente de sesiones de usuarios.
  - Almacenamiento temporal de datos.
  - Procesamiento de eventos en tiempo real.

- La utilización de Redis permite desacoplar parcialmente la carga de trabajo de los sistemas persistentes, contribuyendo a una arquitectura más escalable.

Arquitectura Interna de Redis

- El flujo simplificado puede describirse de la siguiente manera:
  - El cliente envía una petición.
  - Redis recibe la solicitud.
  - El comando es ejecutado sobre la estructura de datos correspondiente.
  - Se actualiza la memoria si es necesario.
  - Se devuelve la respuesta.

Modelo de Datos en Redis

- Aunque suele describirse como una base de datos clave-valor, Redis ofrece un modelo de datos considerablemente más rico que otros sistemas de la misma categoría.

- Cada clave puede contener distintos tipos de estructuras:
  - Strings
  - Hashes
  - Lists
  - Sets
  - Sorted Sets
  - Etc

Modelo de Datos en Redis

- Esta diversidad permite resolver múltiples problemas sin necesidad de incorporar sistemas adicionales.

- Por ejemplo, un sistema de comercio electrónico podría utilizar:
  - Hashes para almacenar productos.
  - Lists para colas de procesamiento.
  - Sorted Sets para rankings.
  - Strings para sesiones de usuario.

- Esta capacidad convierte a Redis en una plataforma extremadamente flexible para aplicaciones de alta demanda.

Ejemplo: Redis

- Supongamos que una aplicación tiene una base de datos tradicional (MySQL, PostgreSQL, SQL Server, etc.).
- Cuando un usuario pide información, la aplicación consulta la base de datos:
  - Usuario -> aplicación -> bases de datos
- El problema es que acceder a la base de datos puede ser relativamente lento si miles de usuarios hacen consultas al mismo tiempo.
- Redis funciona como una “memoria” que se coloca entre la aplicación y la base de datos.

Ejemplo: Redis

- El objetivo es:
  - Buscar primero en Redis.
    Si el dato existe, devolverlo inmediatamente.
  - Si no existe:
    consultar la base de datos;
    guardar el resultado en Redis;
    devolver la respuesta.

- Supongamos una tienda online, cada vez que alguien entra a ver el producto con codigo 238.

Ejemplo: Redis

- La aplicación consulta PostgreSQL o el motor relacional
  - Si 10.000 usuarios consultan el mismo producto, se ejecutará:

```
SELECT *
FROM productos
WHERE id = 238;
```

- La base de datos recibirá potencialmente 10.000 consultas.

Ejemplo: Redis

- Con Redis:
  - La primera consulta obtiene el dato desde PostgreSQL o el motor relacional que sea y lo guarda.
- Redis:
  - producto:238
- Las siguientes 9.999 consultas se responden desde Redis.
- Resultado:
  - Menos carga en PostgreSQL.
  - Menor tiempo de respuesta.
  - Mejor experiencia para el usuario.

Ejemplo: Redis

- ¿Cómo se almacena información? Redis almacena pares:
  - clave → valor

- Por ejemplo:
  - usuario:1 → Juan
  - usuario:2 → María
  - usuario:3 → Pedro
- Comandos básicos
  - Guardar: HSET usuario:1 "Juan"
  - Obtener: GET usuario:1
    Resultado: Juan
  - Eliminar: DEL usuario:1

Ejemplo: Redis/Java

```
@Autowired
private RedisTemplate<String, Object> redisTemplate;
```

Sin Redis: cada llamada accede a la base de datos.

```
public Producto obtenerProducto(Long id) {

    return productoRepository
            .findById(id)
            .orElseThrow();
}
```

Ejemplo: Redis/Java

Con Redis:

Primera consulta → PostgreSQL o motor relacional

Siguientes consultas → Redis.

Ejemplo: Redis/Java

Con Redis:

```
public Producto obtenerProducto(Long id) {

    String key = "producto:" + id;

    Producto producto =
        (Producto) redisTemplate
            .opsForValue()
            .get(key);

    if(producto != null) {
        return producto;
    }

    producto = productoRepository
        .findById(id)
        .orElseThrow();

    redisTemplate
        .opsForValue()
        .set(key, producto);

    return producto;
}
```

Ejemplo: Redis/Java - Invalidar Cache

- Supongamos se tiene una tabla de productos:

ID	Nombre	Precio
1	Laptop	1000
2	Keyboard	40
3	Mouse	12

- Y Redis almacena el producto 1

```
producto:1
{
   id:1,
   nombre:"Laptop",
   precio:1000
}
```

Ejemplo: Redis/Java - Invalidar Cache

- Ejemplo de flujo de ejecución:
  - Primera consulta de Usuario llama a api: GET /productos/1
  - En la aplicación:

    Redis: producto:1 -> No existe (Redis Miss)
  - Entonces en la aplicación se realiza lo siguiente:

    Redis Miss -> PostgreSQL -> Producto -> Almacena en Redis

Ejemplo: Redis/Java - Invalidar Cache

- En Redis queda:

```
producto:1
{
  id:1,
  nombre:"Laptop",
  precio:1000
}
```

Ejemplo: Redis/Java - Invalidar Cache

  - Segunda consulta de Usuario llama a api: GET /productos/1
  - Ahora: Redis HIT
  - No se consulta la base de datos.
- Supongamos que alguien actualiza el valor de la Laptop de 1000 a 1200. En PostgreSQL queda así:

ID	Nombre	Precio
1	Laptop	1200
2	Keyboard	40
3	Mouse	12

Ejemplo: Redis/Java - Invalidar Cache

- Pero Redis sigue teniendo:1000 porque nadie lo actualizó.
- Entonces una solución posible es invalidar el cache cuando se actualiza el producto para que lo vuelva a refrescar en Redis
  - Se actualiza PostgreSQL.
  - Se elimina Redis.
  - La próxima lectura vuelve a consultar la BD.

  Actualizar -> DELETE producto:1 -> Redis vacío

Ejemplo: Redis/Java - Invalidar Cache

- Proxima consulta: GET /productos/1

  Redis MISS ->PostgreSQL -> Redis

- Ahora Redis tendrá: precio=1200

Ejemplo: Redis/Java - Invalidar Cache

```
public Producto actualizarProducto( Long id, ProductoDTO
dto) {
 Producto producto =
productoRepository.findById(id).orElseThrow();
    producto.setPrecio(dto.getPrecio());
    productoRepository.save(producto);
    redisTemplate.delete( "producto:" + id);
    return producto;
}
```

Ejemplo: Redis/Java - Actualizar Cache

- Otra solución posible es actualizar el cache cuando se actualiza el producto para que a Redis también le quede actualizado
  - Se actualiza PostgreSQL.
  - Se actualiza Redis.

Ejemplo: Redis/Java - Actualizar Cache

- Otra solución: actualizar Redis inmediatamente

```
public Producto actualizarProducto(Long id, ProductoDTO dto) {
    Producto producto
=productoRepository.findById(id).orElseThrow();
    producto.setPrecio(dto.getPrecio());
    producto =productoRepository.save(producto);
    redisTemplate.opsForValue().set("producto:" + id,producto);
    return producto;
}
```

Ejemplo: Redis/Java - TTL

- Otra solución: Expiración automática (TTL)

  redisTemplate.opsForValue().set("producto:1",producto, Duration.ofMinutes(10));

- Conceptualmente el producto:1 expira en 10 minutos
  - ¿Qué pasa? Durante 10 minutos: Redis HIT
  - Después: Redis elimina automáticamente la clave
  - Próxima consulta:

    Redis MISS->PostgreSQL -> Redis

Ejemplo: Redis/Java - Invalidar Cache (3)

- Si alguien cambia el precio del producto a 1200 Redis seguirá mostrando 1000 por 10 minutos
- Pero al vencer el TTL lo eliminará del caché hasta que se consulte nuevamente

Persistencia en Redis

- Aunque Redis almacena información en memoria, dispone de mecanismos para persistir datos en disco.
- RDB (Redis Database Snapshot):
  - Genera imágenes completas del estado del sistema en momentos determinados.
- Ventajas:
  - Recuperación rápida.
  - Archivos compactos.
- Desventajas:
  - Posible pérdida de información reciente.

Persistencia en Redis

- AOF (Append Only File):
  - También registra secuencialmente todas las operaciones ejecutadas.
- Ventajas:
  - Mayor durabilidad.
  - Menor pérdida de datos.
- Desventajas:
  - Mayor consumo de almacenamiento.
- La combinación de ambos mecanismos permite equilibrar rendimiento y confiabilidad.